Week 4: Applied Machine Learning

Python 200 | Mentor Session

What This Week Covers

Students have trained and evaluated classifiers for two weeks. This week adds the tools that make classifiers actually usable.

- **ROC curves and AUC** — evaluate a classifier across every threshold, not just 0.5
- **GridSearchCV** — find the best hyperparameters without data leakage
- **joblib** — save a trained model to disk and load it in a separate script

The running example is a weather classifier that predicts whether daily conditions are good for an outdoor run. By the end of the week, students have a trained, tuned model saved to disk — the same file they will reuse as a pipeline component in weeks 9–10.

The Threshold Problem

Every classifier outputs a probability. We convert it to a prediction by asking: is the probability above 0.5?

0.5 is a convention — not something the model learned, and not always the right choice.

- A day with probability 0.48 gets labeled "bad"
- A day with probability 0.51 gets labeled "good"
- At the same threshold: missing a great running day and recommending a run in a storm are treated as equally bad

The right threshold depends on which type of error costs more in your application. There is no single correct answer.

> **Discussion:** You're building a weather app that suggests whether to go for a run. Is a false positive (recommending a run on a bad day) or a false negative (missing a good day) worse? How would that change your threshold?

TPR and FPR

Two rates describe classifier behavior at any threshold.

True Positive Rate (TPR / Recall) — of all truly good running days, what fraction did the model correctly identify?

False Positive Rate (FPR) — of all truly bad days, what fraction did the model incorrectly call "good"?

Lowering the threshold → model calls more days "good" → TPR goes up, but so does FPR.

Every threshold value produces exactly one (FPR, TPR) pair. The ROC curve plots all of them at once.

The ROC Curve

The ROC curve (Receiver Operating Characteristic) plots FPR on the x-axis and TPR on the y-axis, sweeping the threshold from 0 to 1.

Two reference points anchor every ROC curve:

- **Bottom-left (0, 0)** — threshold = 1.0: model predicts nothing positive. No false alarms, but catches nothing.
- **Top-right (1, 1)** — threshold = 0.0: model predicts everything positive. Catches all good days, but also flags every bad one.

The **diagonal** is a random classifier — no better than a coin flip.

A useful classifier bows toward the top-left corner (high TPR, low FPR).

AUC — One Number for the Whole Curve

AUC (Area Under the Curve) summarizes the ROC curve as a single number.

- Ranges from **0.5** (random) to **1.0** (perfect)
- Interpretation: the probability that the model ranks a random positive above a random negative
- AUC 0.93 → the model correctly ranks 93% of (good day, bad day) pairs

AUC is **threshold-independent** — it measures discrimination ability regardless of where you set the cutoff. Use it during model selection. Once you've chosen a model and threshold, evaluate with precision, recall, and F1 at that operating point.

> **Discussion:** Two classifiers both get 85% accuracy on the test set. Model A has AUC 0.91, Model B has AUC 0.74. Which would you choose to develop further? Why might accuracy mislead you here?

Lesson 1 Demo: ROC & AUC

Open: Lesson 1 → weather classifier code

Focus on	Skim for now
How the label is engineered from raw weather features	Exact threshold values in `label_running_day`
The ROC curve shape — how far it bows from the diagonal	`roc_curve()` internals
The AUC score and what it tells you about this classifier	`predict_proba()` plumbing
The threshold table: how TPR and FPR shift as you move the cutoff	

Try manually adjusting the threshold. Watch what happens to the number of recommended days.

> **Discussion:** The weather classifier has unusually high AUC because the labels were defined from those exact features. In what real-world situation might you get low AUC even with a well-built model?

The Hyperparameter Tuning Problem

Every model has hyperparameters: `k` in KNN, `C` in logistic regression, `max_depth` in a decision tree.

Manual tuning breaks down in two ways:

1. **Combinatorial explosion** — two parameters with 5 values each = 25 combinations to test by hand
2. **Data leakage** — if you look at test-set performance to choose hyperparameters, the test set is no longer a fresh holdout. The reported score will be optimistically wrong.

The fix: tune on the training data only using **cross-validation**. Reserve the test set for a single final evaluation.

> **Discussion:** You try `C=1.0` on the test set (89% accuracy), then `C=10.0` (91%), and report 91%. What is wrong with this, even though you're picking the better model?

GridSearchCV

`GridSearchCV` automates hyperparameter search with cross-validation.

You define the grid. It trains a model for every combination across every fold, then returns the best configuration — without touching the test set.

```
grid_search = GridSearchCV(
    estimator=LogisticRegression(max_iter=1000),
    param_grid={"C": [0.001, 0.01, 0.1, 1.0, 10.0]},
    cv=5,
    scoring="roc_auc",
)
grid_search.fit(X_train_scaled, y_train)

print(grid_search.best_params_)   # e.g. {'C': 1.0}
print(grid_search.best_score_)    # mean CV AUC across folds
```

`best_estimator_` is the winning model, already retrained on the full training set. Call `.predict()` directly on it.

Pipeline + GridSearchCV: The Right Way

Scaling *before* passing to GridSearchCV is a subtle data leak — the scaler has seen the held-out fold.

The fix: bundle scaler and model into a `Pipeline` and pass raw data. Scaling happens inside each fold.

```
pipe = Pipeline([
    ("scaler", StandardScaler()),
    ("clf",    LogisticRegression(max_iter=1000)),
])

param_grid = {"clf__C": [0.001, 0.01, 0.1, 1.0, 10.0]}

grid_search = GridSearchCV(pipe, param_grid, cv=5, scoring="roc_auc")
grid_search.fit(X_train, y_train)   # raw data — pipeline handles scaling
```

`clf__C` uses the `stepname__parametername` convention to reach inside the pipeline.

> **Discussion:** The double-underscore syntax (`clf__C`) surprises students. Ask: can someone explain in plain English what `clf__C` refers to?

Lesson 2 Demo: GridSearchCV

Open: Lesson 2 → GridSearchCV notebook

Focus on	Skim for now
The `param_grid` dictionary and how to read it	`cv_results_` internals
Pipeline parameter names with `__` separator	`RandomizedSearchCV`
`best_estimator_` — what it is and how to use it	Searching across model types
`cv_results_` mean and std per parameter value	

Run the grid search twice: once on pre-scaled data, once via Pipeline. Compare the CV scores.

> **Discussion:** You have 6 values of `C` and 4 values of `max_depth` and you're running `cv=5`. How many total model fits does the search require?

Training Once, Predicting Many Times

Every script so far follows this pattern: load data → train → evaluate → done. Training and prediction happen in the same run.

In production, they are separated — sometimes by hours, sometimes by months:

- A model is **trained offline**, on historical data, at regular intervals
- The trained model is **saved to disk**
- A separate script **loads the model** and predicts on new data — fast, lightweight, no retraining

This separation means a model can be shared across scripts, deployed to different environments, and rolled back to an earlier version if something goes wrong.

`joblib` is the standard library for saving and loading scikit-learn models.

Why Save the Full Pipeline

Saving just the model is a common mistake that produces **silently wrong predictions**.

The scaler is learned state: it stores the training set's mean and standard deviation for each feature. If you throw it away, predictions on raw data will be wrong — with no error message.

Saving the full Pipeline solves this:

- One file: scaler and model bundled together
- Calling `.predict()` on the pipeline with raw data automatically applies the scaler first
- The prediction script needs no knowledge of how the data was preprocessed

> **Discussion:** A colleague loads your saved `LogisticRegression` (not the Pipeline) and calls `.predict(X_new)` on raw data. The code runs without errors. Why should you still be worried?

Save and Load

```
import joblib

# Training script — run once
joblib.dump(best_pipe, "models/weather_classifier.pkl")
print("Model saved.")

# Prediction script — run whenever you need predictions
clf = joblib.load("models/weather_classifier.pkl")
predictions  = clf.predict(new_days)       # raw data, no scaling needed
probabilities = clf.predict_proba(new_days)[:, 1]
```

Always save a metadata file alongside the `.pkl`:

- Python and scikit-learn versions (serialized models are version-sensitive)
- Feature names and their expected order
- What the model was trained on (data source, date range, label thresholds)

Lesson 3 Demo: joblib

Open: Lesson 3 → model persistence notebook

Focus on	Skim for now
`joblib.dump` and `joblib.load` syntax	Binary serialization format
Calling `.predict()` on a loaded Pipeline with raw data	Version pinning in `requirements.txt`
The metadata file and what to include	

Run the training script. Then open a fresh notebook, load the `.pkl`, and make predictions — without importing any training code.

> **Discussion:** What would happen if someone ran `predict_weather.py` before `train_weather_classifier.py`? How could you make that error message more helpful?

Week 4 Wrap-Up

- **Threshold problem** — 0.5 is a convention; the right cutoff depends on the cost of each error type
- **ROC curve** — plots TPR vs. FPR across every threshold; good classifiers bow toward the top-left
- **AUC** — threshold-independent summary of discrimination ability; use during model selection
- **GridSearchCV** — systematic, CV-based search; pass a Pipeline to avoid data leakage inside the folds
- **joblib** — serialize the full Pipeline (scaler + model) to avoid silent errors on raw input
- **Metadata** — document Python/sklearn versions, feature names, and training provenance

Takeaway: Applied ML isn't just about picking an algorithm — it's about tuning it correctly, evaluating without leaking data, and building models that can actually be used outside the training script. The weather classifier you saved today will reappear as a live component in a cloud data pipeline in weeks 9–10.